CN 2026-04-22

Approssimazione di integrali definiti

- **Problema:**

$$I(f) = \int_a^b f(x)\,dx \qquad -\infty < a < b < +\infty$$

$$f \in C[a,b]$$

- $I(f) \approx I_n(f) = \int_a^b p(x)\,dx$

$p(x) \in \Pi_n$: $p(x_i) = f(x_i) = f_i, \quad i = 0, \dots, n$

$x_i = a + i\,h, \qquad h = \frac{b-a}{n}$.

- $I_n(f) = h \sum_{i=0}^{n} c_{in} f_i$ (formule di Newton-Cotes)

$$I(f) - I_n(f) = \gamma_n f^{(n+\mu)}(\xi) \left(\frac{b-a}{n}\right)^{n+\mu+1}$$

$$\mu = \begin{cases} 1, & n \text{ dispari} \\ 2, & n \text{ pari} \end{cases}$$

- Per motivi di condizionamento,

$$n \in \{1, \dots, 7, 8\}$$

- **Formule composite di Newton-Cotes**

Se $n = k\ell$, $\ell \in \mathbb{N}$, allora possiamo applicare ℓ volte la formula di Newton-Cotes di

grado k per approssimare $I(f)$. In questo modo otteniamo la formula composita di grado k, $I_k^{(n)}(f)$. Così facendo, considerando $k \in \{1, -, 7, 9\}$, si ottiene che l'errore decresce, al crescere di n, essendo k fissato:

(1) $I(f) - I_k^{(n)}(f) =$ $\xi \in [a,b]$

$$\gamma_k \left(\frac{b-a}{k}\right) f^{(k+\mu)}(\xi) \left(\frac{b-a}{n}\right)^{k+\mu} \longrightarrow 0 \quad (n \to \infty)$$

Stima dell'errore di quadratura

Vediamo come, dalla (1), si possa ottenere, a costo nullo, una stima dell'errore di quadratura.

oss: il costo computazionale verrà misurato in termini di valutazioni funzionali della funzione integranda, $f(x)$, che, pertanto, non devono essere ridondanti.

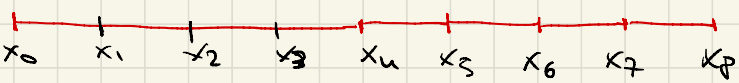
Se supponiamo che, nella (1), n sia

del tipo

$$n = 2\,k\ell\,,$$

ovvero, sia un multiplo pari di k, allora possiamo applicare la formula composita, sia su tutti gli $n+1$ punti, che su quelli di indice pari:

es:



Simpson $k=2$ => $n=8$

Possiamo quindi applicare la formula composita sia su tutti i punti, che solo su quelli pari.

Così facendo, dalla (1) otteniamo che, per un opportuno $\hat{\xi} \in [a,b]$ (generalmente, $\hat{\xi} \neq \xi$) e ponendo $n/2$ al posto di n:

$$(2)\quad I(f) - I_k^{(n/2)}(f) = \nu_k \left(\frac{b-a}{k}\right) f^{(k+\mu)}(\hat{\xi}) \left(\frac{b-a}{n/2}\right)^{k+\mu}$$

ricordiamo che:

$$(1)\quad I(f) - I_k^{(n)}(f) = \nu_k \left(\frac{b-a}{k}\right) f^{(k+\mu)}(\xi) \left(\frac{b-a}{n}\right)^{k+\mu}$$

Facendo la posizione che $\xi \approx \hat{\xi}$,

sottraendo la (1) dalla (2), membro a membro,

otteniamo che:

$$I_k^{(n)}(f) - I_k^{(n/2)}(f) \approx \nu_k \left(\frac{b-a}{k}\right) f^{(k+\mu)}(\xi) \left(\frac{b-a}{n}\right)^{k+\mu} (2^{k+\mu} - 1)$$

Pertanto, dividendo membro a membro per $2^{k+\mu} - 1$, otteniamo che:

$$\hat{E}_k^{(n)}(f) = \frac{I_k^{(n)}(f) - I_k^{(n/2)}(f)}{2^{k+\mu} - 1} \approx \nu_k \left(\frac{b-a}{k}\right) f^{(k+\mu)}(\xi) \left(\frac{b-a}{n}\right)^{k+\mu}$$

$$= I(f) - I_k^{(n)}(f) = E_k^{(n)}(f).$$

Così facendo, otteniamo una stima computazionale dell'errore di quadratura. Chiaramente, questa diviene più accurata al crescere di n.

Ad esempio, se consideriamo

$$I(f) = \int_0^{\pi} \sin(x)\, dx = 2$$

ed utilizziamo le formule composite dei trapezi e di Simpson ($k=1$ e $k=2$, rispettivamente), si ottiene:

n	$I_1^{(n)}$	$E_1^{(n)}$	$\hat{E}_1^{(n)}$	$I_2^{(n)}$	$E_2^{(n)}$	$\hat{E}_2^{(n)}$
2	1.5708	4.2920e-1	5.2360e-01	2.0944	-9.4395e-2	—
4	1.8961	1.0388e-1	1.0844e-01	2.0046	-4.5598e-3	-5.9890e-03
6	1.9541	4.5903e-2	4.6766e-02	2.0009	-8.6319e-4	—
8	1.9742	2.5768e-2	2.6038e-02	2.0003	-2.6917e-4	-2.8604e-04
10	1.9835	1.6476e-2	1.6586e-02	2.0001	-1.0952e-4	—
12	1.9886	1.1436e-2	1.1489e-02	2.0001	-5.2624e-5	-5.4038e-05
14	1.9916	8.3996e-3	8.4279e-03	2.0000	-2.8344e-5	—
16	1.9936	6.4297e-3	6.4462e-03	2.0000	-1.6591e-5	-1.6839e-05
18	1.9949	5.0795e-3	5.0899e-03	2.0000	-1.0348e-5	—
20	1.9959	4.1140e-3	4.1208e-03	2.0000	-6.7844e-6	-6.8489e-06

Possiamo specializzare ulteriormente questa stima dell'errore di quadratura, per ottenere delle formule di quadratura di tipo adattivo.

Il problema è il seguente: talora $I(f)$ è definito da una funzione integranda che è variabile (talora anche molto) in un sotto intervallo dell'intervallo di integrazione.

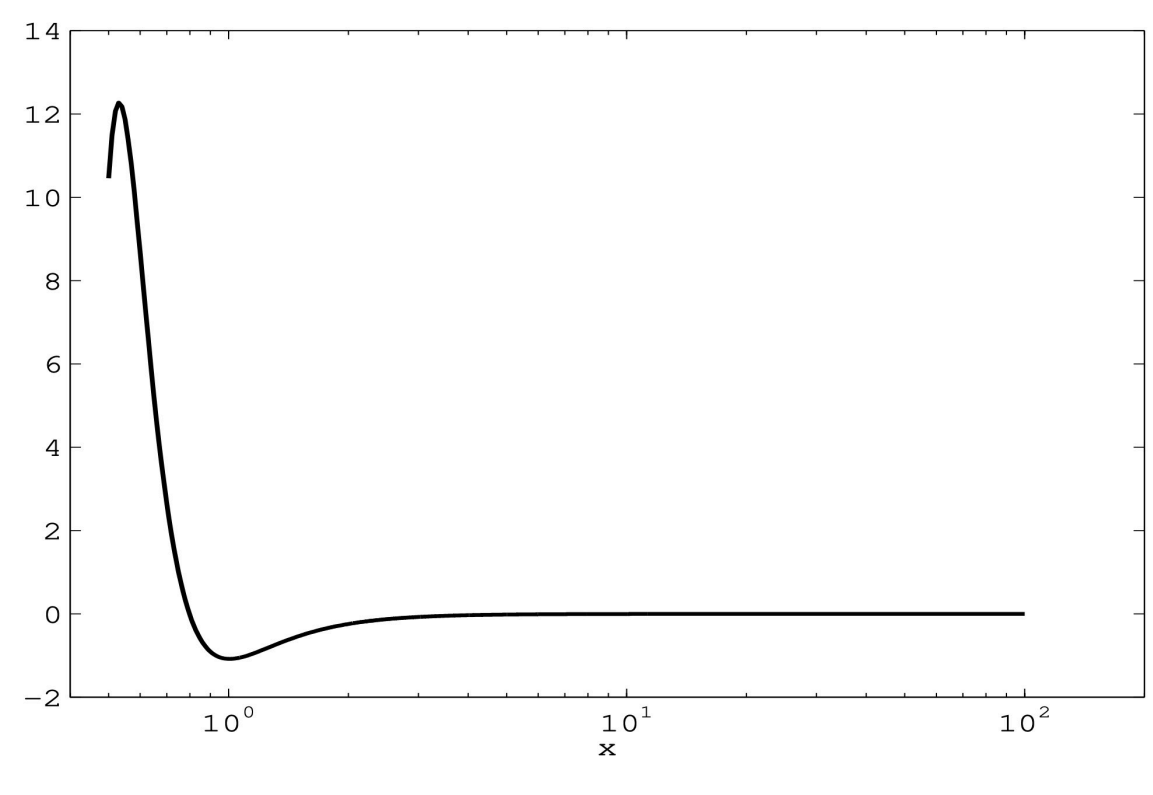
In questo caso, può essere conveniente utilizzare dei punti distribuiti non uniformemente, ma adattati al comportamento locale di $f(x)$. Il fine è, pertanto, quello di soddisfare un prescritto criterio di accuratezza con un numero di punti minore possibile.

Esempio:

la formula è sempre la stessa, cambia COME la usi, non la formula
Il metodo adattivo fa questo: "Non scelgo n prima. Lo decido automaticamente mentre calcolo."

la stima dell'errore? Serve solo a capire se una zona è "difficile"

$$\int_{\frac{1}{2}}^{100} -2x^{-3}\cos(x^{-2})\,dx \equiv \sin(10^{-4}) - \sin(4),$$


14
12
10
8
6
4
2
0
−2
10^0
10^1
10^2
x

Illustriamo questo concetto con la formula dei trapezi, di cui esaminiamo anche la implementazione, sebbene la cosa si estenda facilmente al caso di formule di base più accurate.

Problema: calcolare $I(f)$ con una tolleranza tol prescritta.

Applicando la formula dei trapezi su $[a,b]$ e quella composita con $n=2$, otteniamo che:

$$I1 = \frac{b-a}{2}\left(f(a)+f(b)\right);$$ grossolana (1 trapezio)

$$I2 = \frac{b-a}{4}\left(f(a)+2f(x1)+f(b)\right);$$ più precisa (2 trapezi)

con $x1 = (a+b)/2$; [a, x1], [x1,b]

Abbiamo visto che la stima dell'errore è ottenibile come:

$$E12 = abs(I2-I1)/3;$$ 2^2-1=3

A questo punto:

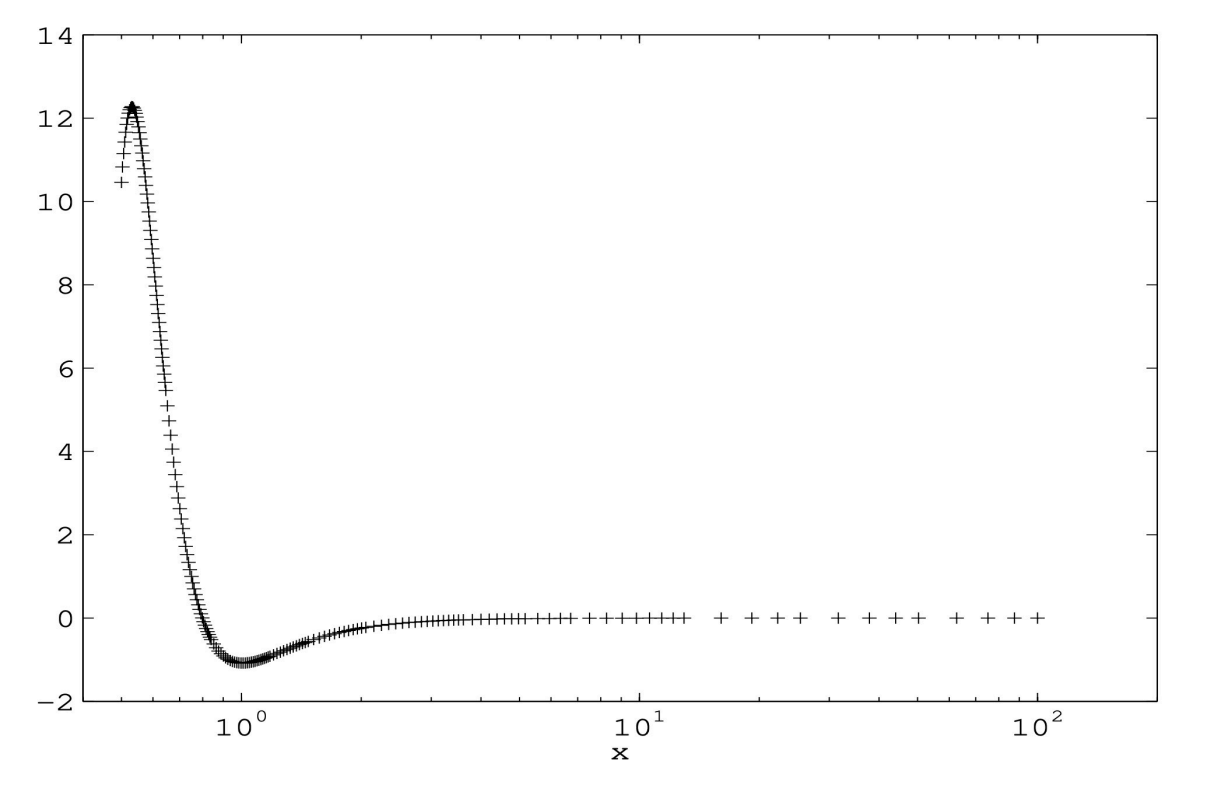
1) se $E12 \leq tol$, ho finito $\Rightarrow I2$

2) altrimenti, riapplico la stessa procedura su $[a, x1]$ e $[x1, b]$, con tolleranza $tol/2$;

L'accortezza nell'implementare questa procedura è quella di evitare valutazioni ridondanti della funzione integranda.

Infatti, nel caso 2) abbiamo già calcolato $f(a)$, $f(x1)$ e $f(b)$ che, pertanto, non vanno ricalcolati.

Ad esempio, per l'esempio visto in precedenza con $tol = 10^{-3}$ (mi pare), si ottiene quanto segue, mediante una semplice implementazione Matlab, che esaminiamo.


14
12
10
8
6
4
2
0
−2
10^0
10^1
10^2
x

```
function I2 = trapead ( fun, a, b, tol, fa, fb )
%
%  I2 = trapead ( fun, a, b, tol)
%
%  Calcola l'integrale di fun in [a,b]
%  con tolleranza tol
%
%  ...

< Controlli di consistenza >

if nargin == 4
    fa = fun(a);
    fb = fun(b);
end
x1 = (a+b)/2;
f1 = fun(x1);
h = (b-a)/2;
I1 = (fa+fb)*h;
I2 = (fa + 2*f1 + fb)*h/2;

err = abs(I2-I1)/3;

if err > tol

   I2 = trapead ( fun, a, x1, tol/2, fa, f1) +...
        trapead (fun, x1, b, tol/2, f1, fb);

end

return.
```